

Classification with k-nearest neighbors

Types of Machine Learning Algorithms

Broadly speaking, there are two types of machine learning algorithms.

Supervised methods: Data items have labels, and we want to learn a function that correctly assigns labels to new data items.

Unsupervised methods: Data items do not have labels, and we want to learn a function that extracts important patterns from the data.

In both cases, each data item is a vector $\mathbf{x}$.

The entries of $\mathbf{x}$ are called features.

Supervised Learning

The Setup

- **Given:** Training data $\{(\mathbf{x}_i, y_i) | i = 1, 2, \dots, N\}$: Vectors paired with labels
- **Goal:** Predict y for new, unseen $\mathbf{x}$
- **Assumption:** Labels depend on features (otherwise, why bother?)

In this class, we think of the data $\mathbf{x}$ as a vector. We use the term "features" to refer to the components of $\mathbf{x}$.

Terminology:

- $\mathbf{x} = \textit{features}$ (independent variables)
- $y = \textit{target}$ (dependent variable)

Key principles:

- There's a set of possible prediction rules. *Learning* means finding the best one.
- The rule that works on training data should also work on new data. This is called *generalization*.

We don't just want to memorize — we want to learn patterns that transfer!

Regression vs Classification

Regression: y is continuous (a number)

- **Example:** Predict house price from sq footage, bedrooms, location: in predicting housing prices, the features $x_1, x_2, x_3 \dots$ could be a vector containing lot size, square footage, number of bathrooms, etc. of a collection of houses, and y_i could be the sale price of house i .
- **Success** = predictions are close to true values

Classification: y is discrete (a category)

- **Example:** Image recognition: the features x could be a vector that represents the pixels of the image, and could be a label such as "tiger," "tree," etc.
- **Success** = predictions are correct most of the time

Which type is each:

- predicting tomorrow's temperature?
- Diagnosing a disease?
- Estimating ad revenue?
- Predicting star ratings? Election outcomes?

Parametric vs Non-Parametric Models

Another way in which different machine learning models can vary (whether supervised or unsupervised) is in the number of training parameters that exist.

- If the model has a **fixed number** of parameters, no matter the size of the training dataset, then it is called **parametric**.
- If the number of parameters **grows with the data**, the model is called **nonparametric**.

Parametric models have the advantage of

- Often being faster to use,
- The disadvantage of making strong assumptions about the nature of data distributions.

Nonparametric models are

- More flexible,
- Can be computationally intractable for large datasets.

***k*-Nearest Neighbors**

The classic example of a **supervised, nonparametric** classifier is called ***k*-Nearest Neighbors**.

Recall:

We learned how to compute the distance between two vectors:

$$\mathbf{dist}(\mathbf{u}, \mathbf{v}) = \sqrt{(\mathbf{v}_1 - \mathbf{u}_1)^2 + (\mathbf{v}_2 - \mathbf{u}_2)^2 + \cdots + (\mathbf{v}_n - \mathbf{u}_n)^2}$$

Today we'll use this to build our first machine learning algorithm!

The key idea: similar data points should have similar labels.

If we want to predict the label for a new point, we can look at the labels of nearby points.

***k*-Nearest Neighbors**

***k*-Nearest Neighbors** (*k*-NN) is one of the simplest machine learning algorithms.

The idea: “Ask your neighbors!”

- Given a new data point , find the closest points in the training data
- Use their labels to make a prediction

By “closest” we mean smallest ***Euclidean distance*** — We already know what that is!

k-NN can be used for both classification and regression:

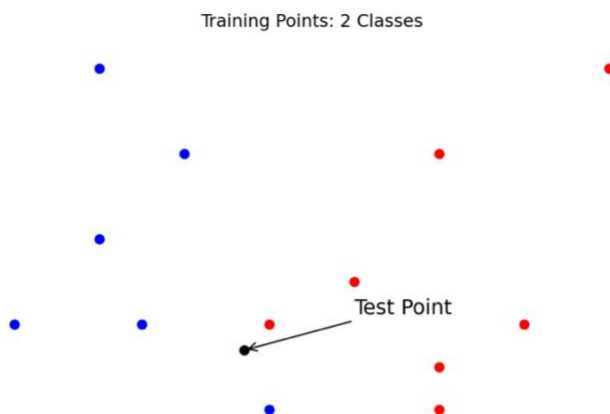
- ***Classification:*** predict the majority label among the neighbors
- ***Regression:*** predict the average value among the neighbors

k-Nearest Neighbors

Here is an example of a training dataset $\{(\mathbf{x}_i, y_i) | i = 1, 2, \dots, N\}$ where the feature vectors are each 2-dimensional, and the *label* is either '*blue*' or '*red*'.

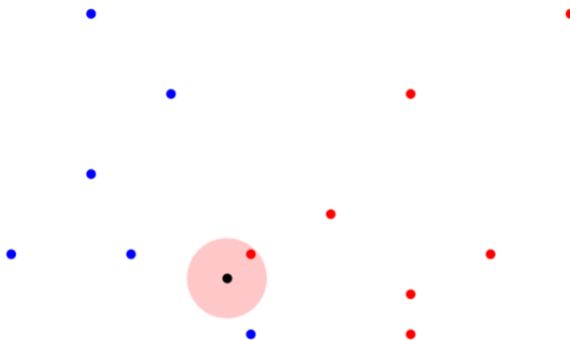


The idea of the *k*-Nearest Neighbors classifier is that, at classification time, it simply "looks at" the *k* points in the training set that are nearest to the test input point $\mathbf{x}$, and makes a decision based on the labels on those points.

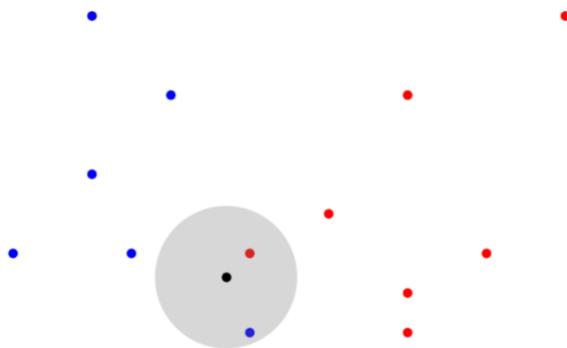


k -Nearest Neighbors

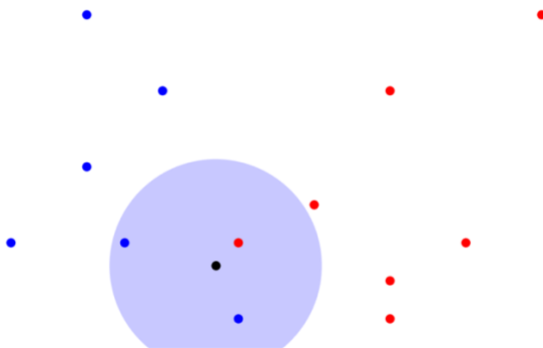
1-Nearest-Neighbor: Classification: Red



2-Nearest-Neighbor



3-Nearest-Neighbor: Classification: Blue



***k*-Nearest Neighbors**

Note that k -Nearest Neighbors can do either ***hard*** or ***soft*** classification.

As a ***hard*** classifier, it returns the majority vote of the labels on the nearest k neighbors (which may be indeterminate).

As a ***soft*** classifier, it returns

$$p(y = c|x, k) = \frac{\text{number of neighbors with label } c}{k}$$

Given training points with labels:

A = (1, 2) → Red, B = (3, 1) → Red, C = (2, 4) → Blue, D = (4, 3) → Blue

What does k -NN predict for the new point X = (2, 2)?

- Using 1-NN?
- Using 3-NN?

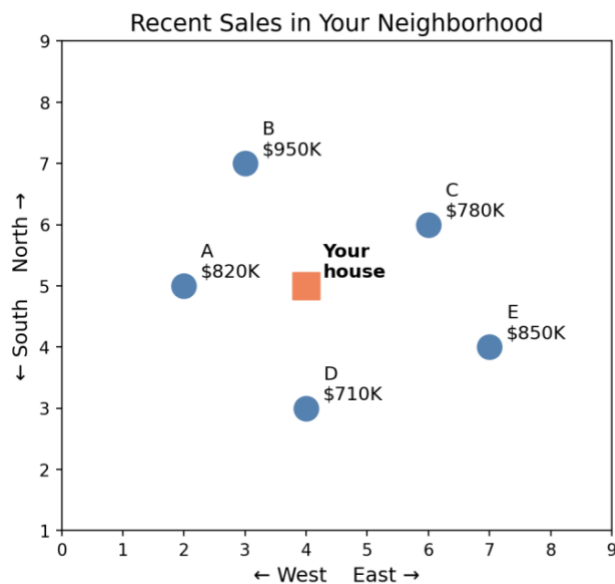
***k*-NN for regression**

k-NN also works for regression (predicting continuous values). Instead of majority vote, take the average of neighbors' values.

Example: Predicting House Prices

You want to sell your house. How do you estimate its value?

Look at what nearby houses sold for recently!



Using $k = 2$, nearests are A and D.

Thus, predict $\frac{\$820K + \$710K}{2} = \$765K$

Wait!!!— should a 1-bedroom condo really be compared to a 5-bedroom mansion just because they're close on the map?

Distance in feature space

Real estate agents consider “comps” — comparable properties:

House	Sq. Feet	Beds	Baths	Price
A	1,800	3	2	\$820K
B	2,200	4	2.5	\$950K
C	1,400	2	1	\$780K
Yours	1,900	3	2	???

Now each house is a vector: $\mathbf{x} = (\text{sq ft, beds, baths, ...})$

If houses are vectors of features, what does "distance" mean?

How would you find the 2 most similar houses to yours?

Can we still use Euclidean distance?

$$\text{dist}(\mathbf{u}, \mathbf{v}) = \sqrt{(v_1 - u_1)^2 + (v_2 - u_2)^2 + \dots + (v_n - u_n)^2}$$

The “nearest neighbors” are houses with similar features, not just similar locations. (What about scaling?!)

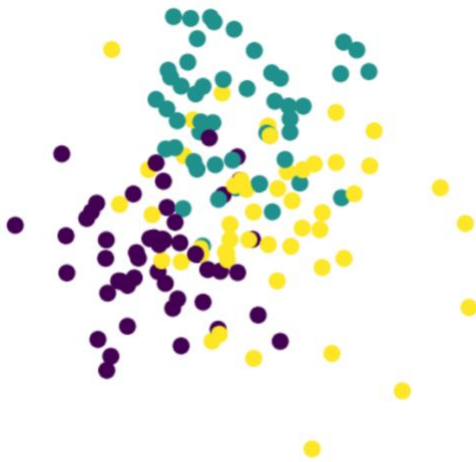
This is exactly what Zillow and Redfin do for their price estimates.

The choice of k matters

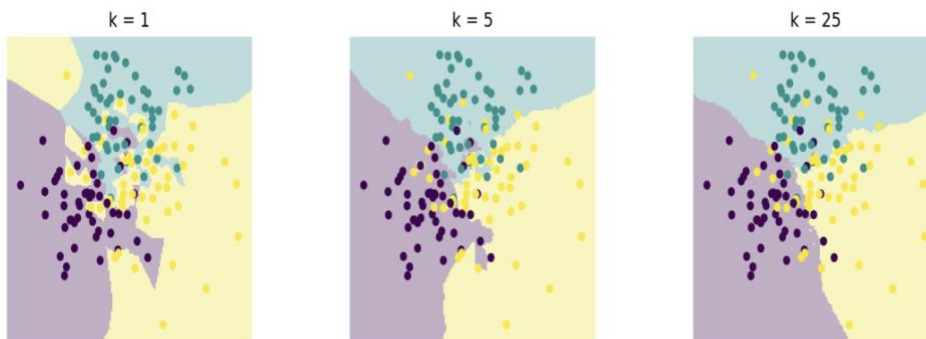
The variable k itself is called a *hyperparameter*. That is: it is a parameter that must be set *before* the model parameters themselves can be learned.

Each choice of k results in a different model. The complexity of the resulting model is therefore controlled by the hyperparameter k .

Consider this dataset where items fall into three classes:



Different values of k will give very different decision boundaries:



- **Small k (like 1):** Very sensitive to noise, complex decision boundaries
- **Large k (like 25):** Smoother boundaries, but may miss local patterns

Overfitting: When k is too small

When $k = 1$, the model memorizes every training point perfectly.

Think: If $k = 1$ gets 100% accuracy on training data, is that good?

No! This is called *Overfitting*.

- The model fits the training data too well
- It learns the noise, not just the signal
- It will perform poorly on new data

The goal isn't to memorize — it's to *generalize* to data we haven't seen yet.

Underfitting: When k is too large

What if we set k to be almost the number of points (50/60)?



With $k = 50$, the model is too simple - it ignores the real structure in the data.

This is *underfitting*.

We need k that's "just right":

- *Too small: Overfitting (memorizes noise)*
- *Too large: Underfitting (misses real patterns)*
- *Just right: Captures signal, ignores noise*

Challenge: Finding the right k